МИНЦИФРЫ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Институт информатики и вычислительной техники
Кафедра Вычислительных Систем
(Каф. Вс)

ПРАКТИЧЕСКАЯ РАБОТА №8
по дисциплине
«Информатика»

по теме
МОДЕЛИРОВАНИЕ АРХИТЕКТУРЫ ФОН НЕЙМАНА НА ЯЗЫКЕ ПРОГРАММИРОВАНИЯ СИ

Вариант 0,5
Уровень «ОТЛИЧНО»

Выполнил:
студент гр. ИС-642
Ковайкин Д.В.

Проверил:
ассистент каф. ВС
Корнилов А.А.

Новосибирск, 2026

СОДЕРЖАНИЕ

1. ЦЕЛЬ РАБОТЫ.....3
2. КРАТКИЕ ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АРХИТЕКТУРЫ ФОН НЕЙМАНА 4
3. ПРОЕКТИРОВАНИЕ ВИРТУАЛЬНОЙ МАШИНЫ.....5
3.1. Организация памяти.....5
3.2. Регистры процессора.....5
3.3. Система команд.....6
3.4. Общая схема выполнения.....6
4. РЕАЛИЗАЦИЯ ПРОГРАММЫ.....8
4.1. Загрузка программы и инициализация.....8
4.2. Цикл выборки и выполнения команд.....8
4.3. Обработка ошибок и вывод состояния.....9
5. ТЕСТИРОВАНИЕ.....11
5.1. Основной тест: арифметические операции и переходы.....11
5.2. Проверка выхода адреса операнда за границы памяти.....12
5.3. Проверка неизвестной команды.....13
ВЫВОД.....15
ПРИЛОЖЕНИЕ А. ПОЛНЫЙ ЛИСТИНГ ПРОГРАММЫ.....16

1. ЦЕЛЬ РАБОТЫ

Изучить основные принципы архитектуры фон Неймана и реализовать их программную модель на языке Си. Разработать виртуальную машину с общей памятью для хранения команд и данных, набором регистров процессора и аккумулятором, поддерживающую выполнение арифметических операций, безусловных и условных переходов.

В ходе работы необходимо организовать загрузку команд из текстового файла, реализовать цикл выборки и выполнения инструкций, предусмотреть завершение программы по команде HALT и проверить работу виртуального процессора на корректных и ошибочных входных данных.

2. КРАТКИЕ ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АРХИТЕКТУРЫ ФОН НЕЙМАНА

Архитектура фон Неймана — принцип построения вычислительной системы, в которой команды программы и обрабатываемые ими данные хранятся в общей памяти. Процессор последовательно извлекает команды из памяти, интерпретирует их и выполняет. Такой подход лежит в основе большинства универсальных компьютеров.

К основным компонентам классической модели относятся центральное устройство обработки информации, память и устройства ввода-вывода. В рамках данной работы устройства ввода-вывода представлены загрузкой программы из файла и выводом состояния виртуального процессора в консоль. Центральный процессор моделируется программно с помощью массива регистров и цикла исполнения.

Выполнение программы обычно описывается циклом «выборка - декодирование - выполнение». Сначала процессор обращается к ячейке памяти, адрес которой хранится в счётчике команд (PC), затем определяет тип инструкции и выполняет соответствующее действие. После этого PC изменяется так, чтобы указывать на следующую команду. Для команд перехода изменение PC определяется адресом перехода.

Важное свойство рассматриваемой модели - единое адресное пространство для команд и данных. Поэтому одна и та же область памяти может содержать машинные инструкции, их операнды и значения, используемые арифметическими командами. Интерпретация содержимого ячейки зависит от текущей выполняемой инструкции.

3. ПРОЕКТИРОВАНИЕ ВИРТУАЛЬНОЙ МАШИНЫ

3.1. Организация памяти

Для моделирования памяти используется массив целых чисел memory размером 256 элементов. Каждая ячейка имеет адрес от 0 до 255. В начале файла program.txt располагается значение вектора прерываний: оно загружается в memory[0]. В используемой тестовой программе выполнение начинается с адреса 6, поэтому первые шесть ячеек отведены под служебное значение и данные.

Команды и их операнды располагаются последовательно. Для команд, которым требуется операнд, сначала записывается код команды, а в следующей ячейке — значение операнда. Команды LOAD, ADD и SUB трактуют операнд как адрес ячейки с данными; команды JUMP и JUMP_IF_ZERO — как адрес следующей инструкции.

Таблица 1 - Назначение областей памяти в тестовой программе

Адреса	**Назначение**
0	Вектор прерываний (значение memory[0])
1–5	Область данных и служебные ячейки
6 и далее	Последовательность команд и их операндов
0–255	Полный диапазон адресов доступной памяти

3.2. Регистры процессора

Состояние виртуального процессора хранится в массиве cpu_state из восьми целочисленных элементов. Использование одного массива позволяет компактно представить основные регистры и служебные признаки процессора.

Таблица 2 - Назначение элементов массива cpu_state

Индекс	Обозначение	Назначение
0	PC	Счётчик команд - адрес текущей инструкции
1	ACC	Аккумулятор - хранение промежуточного и итогового результата
2	IR	Регистр команд - код текущей инструкции
3	IV	Вектор прерываний, загружаемый из

		memory[0]
4	STOP	Флаг остановки виртуального процессора
5	ERROR	Код последней обнаруженной ошибки
6–7	Резерв	Зарезервированные элементы для возможного расширения

3.3. Система команд

В соответствии с вариантом реализованы арифметические команды LOAD, ADD и SUB, команда завершения HALT, а также команды переходов, необходимые для уровня «отлично». Все коды команд представлены целыми числами.

Таблица 3 - Система команд виртуального процессора

Код	Мнемоника	Действие
0	HALT	Завершить выполнение программы
1	LOAD	Загрузить в ACC значение из ячейки memory[operand]
2	ADD	Прибавить к ACC значение из ячейки memory[operand]
3	SUB	Вычесть из ACC значение из ячейки memory[operand]
4	JUMP	Безусловно перейти к адресу operand
5	JUMP_IF_ZERO	Перейти к адресу operand, если ACC равен нулю

Для обычных команд с операндом длина инструкции составляет две ячейки памяти: код операции и операнд. После выполнения LOAD, ADD или SUB счётчик команд увеличивается на два. Команда JUMP непосредственно устанавливает PC в адрес перехода. Команда JUMP_IF_ZERO также занимает две ячейки: если ACC равен нулю, выполняется переход, иначе PC увеличивается на два. HALT не требует операнда.

3.4. Общая схема выполнения

Работу виртуальной машины можно представить следующей последовательностью: загрузка файла в память → инициализация регистров → чтение команды по адресу PC → чтение операнда (если он требуется) →

выполнение команды → изменение PC → повтор цикла до HALT или обнаружения ошибки.

Таблица 4 - Этапы работы программы

Этап	Действие
1. Загрузка	Чтение целых чисел из program.txt в массив memory
2. Инициализация	Установка начального PC, обнуление ACC и служебных регистров
3. Выборка	Чтение кода команды из memory[PC]
4. Декодирование	Определение действия по коду команды
5. Выполнение	Изменение ACC либо PC в зависимости от команды
6. Контроль	Проверка флага остановки и кода ошибки; переход к следующей итерации

4. РЕАЛИЗАЦИЯ ПРОГРАММЫ

4.1. Загрузка программы и инициализация

Для чтения программы используется функция fopen, открывающая файл program.txt в текстовом режиме. Если файл отсутствует или не может быть открыт, программа выводит сообщение File not found! и завершает работу. Функция fscanf считывает целые числа последовательно, независимо от того, разделены они пробелами или переводами строк, и помещает их в массив memory до достижения конца файла или заполнения массива.

Листинг 4.1 — Чтение файла

```
FILE *program = fopen("program.txt", "r");
if (program == NULL)
{
    printf("File not found!\n");
    return 0;
}
while (i < 256 && fscanf(program, "%d", &memory[i]) == 1) i++;
fclose(program);
```

После загрузки выполняется инициализация регистров. Начальное значение PC устанавливается равным 6, аккумулятор и служебные флаги обнуляются, а вектор прерываний получает значение первой ячейки памяти. Таким образом, формат входного файла заранее определяет расположение служебных данных и начало исполняемого участка.

4.2. Цикл выборки и выполнения команд

Основная работа процессора выполняется внутри цикла while. Цикл продолжается, пока флаг остановки и код ошибки равны нулю. Текущий адрес PC сохраняется отдельно для корректного отображения выполненной команды, после чего код инструкции считывается в регистр IR.

Листинг 4.2 — Цикл выборки

```
while (cpu_state[4] == 0 && cpu_state[5] == 0)
{
    int current_pc = cpu_state[0];
    cpu_state[2] = memory[cpu_state[0]];
    /* далее выполняется декодирование и исполнение */
}
```

Если код команды не равен нулю, программа считывает значение из следующей ячейки памяти. Для адреса операнда предусмотрена проверка

выхода за допустимый диапазон. Затем оператор switch выбирает нужную операцию. Например, LOAD копирует значение из указанной ячейки памяти в аккумулятор, ADD прибавляет его к текущему значению аккумулятора, а SUB вычитает его.

Листинг 4.3 — Обработка и выполнение команд

```
switch (cpu_state[2])
{
    case 1: // LOAD
        cpu_state[1] = memory[operand_val];
        break;
    case 2: // ADD
        cpu_state[1] += memory[operand_val];
        break;
    case 3: // SUB
        cpu_state[1] -= memory[operand_val];
        break;
    case 4: // JUMP
        cpu_state[0] = operand_val;
        break;
    case 5: // JUMP_IF_ZERO
        if (cpu_state[1] == 0)
            cpu_state[0] = operand_val;
        else
            cpu_state[0] += 2;
        break;
}
```

Для команды HALT предусмотрена отдельная ветвь: программа выводит состояние аккумулятора и устанавливает флаг остановки. После обычных арифметических команд PC увеличивается на два, поскольку следующая инструкция находится через одну ячейку операнда. Для команд перехода дополнительное увеличение PC не выполняется, так как адрес уже установлен самой командой.

4.3. Обработка ошибок и вывод состояния

В реализации предусмотрены два основных кода ошибки. Код 1 соответствует выходу адреса операнда за границы массива памяти, код 2 — неизвестной команде. При обнаружении ошибки устанавливается соответствующий элемент cpu_state[5], после чего цикл выполнения завершается. В конце работы программа выводит текстовое сообщение, соответствующее коду ошибки.

Листинг 4.4 - Обработка ошибок

```
switch (cpu_state[5])
{
    case 0:
          printf("Program completed successfully.\n");
          break;
    case 1:
          printf("Error: Address of operand out of memory.\n");
          break;
    case 2:
          printf("Error: Invalid command.\n");
          break;
    default:
          printf("Unknown error.\n");
          break;
}
```

После каждой успешно выполненной команды в консоль выводятся адрес команды, её код, значение операнда (если он есть) и текущее значение аккумулятора. Такой формат позволяет проследить изменение состояния виртуального процессора и сопоставить фактическое выполнение с содержимым program.txt.

В исходной реализации при обнаружении ошибки также записывается адрес вектора прерываний в PC. При этом условие основного цикла требует, чтобы код ошибки оставался равным нулю, поэтому после фиксации ошибки цикл прекращается и обработчик по этому адресу не исполняется. В отчёте данное действие рассматривается как сохранение адреса вектора, а не как полноценная обработка прерывания.

5. ТЕСТИРОВАНИЕ

Тестирование проводилось запуском исполняемого файла ./test в среде Fedora. Для проверки использовался текстовый файл program.txt, содержащий последовательность целых чисел. Проверялись арифметические операции, оба варианта поведения условного перехода, безусловный переход, штатное завершение программы и реакции на некорректные входные данные.

5.1. Основной тест: арифметические операции и переходы

В первом тесте проверяется загрузка значения в аккумулятор, вычитание до нуля, условный переход при ненулевом и нулевом аккумуляторе, сложение и безусловный переход к команде HALT.

Листинг 5.1 — Содержимое program.txt для основного теста

```
0
10
3
0
0
0
1
1
5
14
3
1
5
18
1
2
4
22
2
2
4
24
1
1
0
```

Первые шесть значений задают служебную область и данные: memory[0] = 0, memory[1] = 10, memory[2] = 3. Выполнение начинается с адреса 6. Ниже приведён пошаговый разбор фактически выполненных команд.

Таблица 5 - Пошаговая обработка основного теста

PC	Команда	Операнд	Изменение ACC / PC
6	LOAD (1)	1	ACC = memory[1] = 10
8	JUMP_IF_ZERO (5)	14	ACC = 10, условие ложно; PC = 10

10	SUB (3)	1	ACC = 10 − memory[1] = 0
12	JUMP_IF_ZERO (5)	18	ACC = 0, условие истинно; PC = 18
18	ADD (2)	2	ACC = 0 + memory[2] = 3
20	JUMP (4)	24	PC = 24
24	HALT (0)	—	Выполнение завершено; ACC = 3

При первом выполнении JUMP_IF_ZERO аккумулятор равен 10, поэтому переход на адрес 14 не выполняется. После команды SUB аккумулятор становится равным нулю, и следующая условная команда переводит выполнение на адрес 18. Команды, расположенные по адресам 14 и 16, пропускаются. После сложения команда JUMP переводит процессор на адрес 24, где находится HALT.

Листинг 5.2 — Вывод основного теста

```
pc=6: command 1, operand=1, acc=10
pc=8: command 5, operand=14, acc=10
pc=10: command 3, operand=1, acc=0
pc=12: command 5, operand=18, acc=0
pc=18: command 2, operand=2, acc=3
pc=20: command 4, operand=24, acc=3
pc=24: command 0, acc=3
Program completed successfully.
```

Фактический вывод соответствует ожидаемой последовательности. Итоговое значение аккумулятора равно 3, программа завершается штатно. Тест подтверждает правильность работы команд LOAD, ADD, SUB, JUMP и JUMP_IF_ZERO в рассмотренных сценариях.

5.2. Проверка выхода адреса операнда за границы памяти

Во втором тесте проверяется реакция программы на недопустимое значение операнда. Входные данные содержат команду LOAD с адресом 256, который выходит за пределы массива memory, допустимый диапазон которого составляет от 0 до 255.

Листинг 5.3 — Содержимое program.txt для первого теста обработки ошибок

```
0
10
3
0
0
0
1
```

```
256
```

После загрузки процессор начинает выполнение с PC = 6. По адресу 6 находится команда LOAD, а по адресу 7 — операнд 256. При проверке операнда программа обнаруживает, что значение не входит в допустимый диапазон адресов, устанавливает код ошибки 1 и завершает цикл.

Листинг 5.4 — Вывод первого теста обработки ошибок

```
Error: Address of operand out of memory.
```

Полученное сообщение соответствует ожидаемому результату. Проверка подтверждает, что недопустимый адрес операнда обнаруживается до обращения к соответствующей ячейке данных.

5.3. Проверка неизвестной команды

В третьем тесте в область команд помещён код 9, отсутствующий в реализованной системе команд. Ожидаемое поведение — обнаружение неизвестной инструкции, установка кода ошибки 2 и завершение выполнения.

Листинг 5.5 — Содержимое program.txt для второго теста обработки ошибок

```
0
10
3
0
0
0
9
1
```

На начальном адресе 6 процессор считывает код 9. После перехода к обработке команды оператор switch не находит соответствующего варианта и выполняет ветвь default. В ней устанавливается код ошибки 2.

Листинг 5.6 - Вывод второго теста обработки ошибок

```
Error: Invalid command.
```

Фактическое сообщение соответствует ожидаемому. Таким образом, проверена реакция виртуального процессора на код операции, не предусмотренный системой команд.

По результатам тестирования подтверждена работа основных арифметических инструкций, условного и безусловного переходов, команды остановки, а также предусмотренных проверок некорректных адресов операндов и неизвестных команд.

ВЫВОД

В ходе выполнения практической работы была разработана и протестирована упрощённая модель вычислительной системы, основанной на архитектуре фон Неймана. Были реализованы массивы состояния процессора и общей памяти, загрузка команд из файла, выполнение арифметических операций, безусловных и условных переходов, а также обработка ошибок.

В результате тестирования была подтверждена корректность выполнения основных команд и логики изменения счётчика команд в различных ситуациях. Практическая работа позволила наглядно изучить взаимодействие процессора с памятью, принцип последовательного выполнения инструкций и роль регистров в организации вычислительного процесса.

ПРИЛОЖЕНИЕ А. ПОЛНЫЙ ЛИСТИНГ ПРОГРАММЫ

Листинг 6.1 — Полный код программы

```
/*
cpu_state[0] - счётчик команд
cpu_state[1] - аккумулятор
cpu_state[2] - регистр команд
cpu_state[3] - вектор прерываний
cpu_state[4] - флаг остановки
cpu_state[5] - код последней ошибки
cpu_state[6] - резерв
cpu_state[7] - резерв*/

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int cpu_state[8] = {0}; // инициализация массива состояний процессора
    int memory[256] = {0}; // инициализация массива памяти
    int i = 0;

    FILE *program = fopen("program.txt", "r"); //получение указателя на файл и проверка его
наличия
    if (program == NULL)
    {
        printf("File not found!\n");
        return 0;
    }
    while (i < 256 && fscanf(program, "%d", &memory[i]) == 1) i++; // перенос комманд в память
    fclose(program);

    //инициализация cpu_state
    cpu_state[0] = 6;
    cpu_state[1] = 0;
    cpu_state[2] = 0;
    cpu_state[3] = memory[0];
    cpu_state[4] = 0;
    cpu_state[5] = 0;
    while (cpu_state[4] == 0 && cpu_state[5] == 0)
    {
        int current_pc = cpu_state[0];
        cpu_state[2] = memory[cpu_state[0]];

        // Если команда требует операнда (не 0)
        if (cpu_state[2] != 0)
        {
            // Проверка границ для адреса операнда
            if (cpu_state[0] + 1 >= 256 || cpu_state[0] < 0) {
                cpu_state[5] = 1; // Код ошибки
                cpu_state[0] = cpu_state[3]; // Переход на вектор прерываний
                continue; // Цикл заново с нового pc
            }
            int addr = cpu_state[0] + 1; // Адрес ячейки с операндом
            int operand_val = memory[addr]; // Значение операнда (адрес данных)

            // Проверка границ для операнда
            if (operand_val < 0 || operand_val >= 256) {
                cpu_state[5] = 1; // Код ошибки
                cpu_state[0] = cpu_state[3]; // Переход на вектор прерываний
                continue; // Цикл заново с нового pc
            }

            switch (cpu_state[2])
            {
                case 1: // LOAD
                    cpu_state[1] = memory[operand_val];
                    break;
                case 2: // ADD
                    cpu_state[1] += memory[operand_val];
                    break;
                case 3: // SUB
                    cpu_state[1] -= memory[operand_val];
                    break;
```

```
                case 4: // JUMP
                    cpu_state[0] = operand_val;
                    break;
                case 5: // JUMP_IF_ZERO
                    if (cpu_state[1] == 0)
                    {
                        cpu_state[0] = operand_val;
                    }
                    else cpu_state[0] += 2; // Переход к следующей инструкции, если условие не
выполнено
                    break;
                default:
                    cpu_state[5] = 2; // Код ошибки
                    cpu_state[0] = cpu_state[3]; // Переход на вектор прерываний
                    continue; // Цикл заново с нового pc
            }
            printf("pc=%d: command %d, operand=%d, acc=%d\n", current_pc, cpu_state[2],
operand_val, cpu_state[1]);
        }
        // если команда = 0, завершение выполнения
        else // HALT
        {
            printf("pc=%d: command 0, acc=%d\n", current_pc, cpu_state[1]);
            cpu_state[4] = 1;
            continue; // перевод флага работы в 1, переход к следующей итерации для исключения
варианта продолжения работы после смены флага
        }
        //переход к следующей инструкции с шагом 2, пропуская операнды
        cpu_state[0] += (cpu_state[2] != 4 && cpu_state[2] != 5) ? 2 : 0;
    }
    switch (cpu_state[5]) //вывод кода ошибки
    {
        case 0:
            printf("Program completed successfully.\n");
            break;
        case 1:
            printf("Error: Address of operand out of memory.\n");
            break;
        case 2:
            printf("Error: Invalid command.\n");
            break;
        default:
            printf("Unknown error.\n");
            break;
    }

    return 0;
}
```